Palm OS Programming Recipes

Implementing the Required Conduit Entry Points

This recipe shows you how to implement each of the entry points required by a Windows C API-based conduit. HotSync Manager® calls these four entry points in your conduit to execute the synchronization code in your conduit.

This recipe assumes:

- You have installed Microsoft Visual C++ 6.0 and the Palm™ Conduit Development Kit (CDK) version 4.02.
- You have used the Conduit Wizard to build a conduit project that implements "conduit entry points only (no sync logic)."
    - The code segments provided in this recipe are based on the entry points generated by the Conduit Wizard.
    - Values in these entry point functions are maintained in a String Table generated by the Conduit Wizard. You can simply edit these value to provide HotSync Manager with the required values specific to your conduit.

The steps presented in this recipe are:

1. Implement `GetConduitInfo`.
2. Implement `GetConduitName`.
3. Implement `GetConduitVersion`.
4. Implement `OpenConduit`.

1. Implement GetConduitInfo

The `GetConduitInfo` function is called by HotSync Manager to retrieve information about a conduit. HotSync Manager may request three specific types of information from the `GetConduitInfo` function:

- display name of the conduit
- conduit's default action
- MFC version used to build the conduit or whether MFC was used at all

The following sample is an implementation of the `GetConduitInfo` entry point.

```
ExportFunc long GetConduitInfo(ConduitInfoEnum infoType, void *pInArgs,
    void *pOut, DWORD *pdwOutSize)
{
    if (!pOut)
        return CONDERR_INVALID_PTR;
    if (!pdwOutSize)
        return CONDERR_INVALID_OUTSIZE_PTR;

    // infoType specifies one of the following three cases.
    // The remainder of the parameters are defined based
    // on the infoType enum.

    switch (infoType) {
```

```
        case eConduitName:

            if (!pInArgs)
                return CONDERR_INVALID_INARGS_PTR;
            ConduitRequestInfoType *pInfo;
            pInfo = (ConduitRequestInfoType *)pInArgs;
            if ((pInfo->dwVersion != CONDUITREQUESTINFO_VERSION_1) ||
                (pInfo->dwSize != SZ_CONDUITREQUESTINFO))
                return CONDERR_INVALID_INARGS_STRUCT;

            // IDS_CONDUIT_NAME is maintained in the project String Table
            // and contains the conduit name.

            if (!::LoadString((HINSTANCE)hLangInstance, IDS_CONDUIT_NAME,
                   (TCHAR*)pOut, *pdwOutSize))
                return CONDERR_CONDUIT_RESOURCE_FAILURE;
            break;
        case eDefaultAction:
            if (*pdwOutSize != sizeof(eSyncTypes))
                return CONDERR_INVALID_BUFFER_SIZE;
            (*(eSyncTypes*)pOut) = eFast;
            break;
        case eMfcVersion:
            if (*pdwOutSize != sizeof(DWORD))
                return CONDERR_INVALID_BUFFER_SIZE;
            (*(DWORD*)pOut) = MFC_NOT_USED;
            break;
        default:
            return CONDERR_UNSUPPORTED_CONDUITINFO_ENUM;
    }
    return 0;
}
```

2. Implement GetConduitName

The `GetConduitName` function is called by HotSync Manager to provide the name of a conduit. You must implement this function to return a name buffer and the status of the operation. The following code is a sample of the `GetConduitName` function:

```
ExportFunc long GetConduitName(char* pszName,WORD nLen)
{
    long retval = -1;

    // IDS_CONDUIT_NAME is maintained in the project String Table
    // and contains the conduit name.

    if (::LoadString((HINSTANCE)hLangInstance, IDS_CONDUIT_NAME,
        pszName, nLen))
        retval = 0;

    return retval;
}
```

3. Implement GetConduitVersion

The `GetConduitVersion` function is called by HotSync Manager to provide the version of the conduit. The following code is a sample of the `GetConduitVersion` function:

```
ExportFunc DWORD GetConduitVersion()
{
    // GENERIC_CONDUIT_VERSION is defined as 0x00000102
    // in the conduit header file
    //                     0000---- = HIWORD         - reserved
    //                     ----01-- = HIBYTE(LOWORD) - major number
    //                     ------02 = LOBYTE(LOWORD) - minor number

    return GENERIC_CONDUIT_VERSION;
}
```

4. Implement OpenConduit

The `OpenConduit` function is called by HotSync Manager to begin the process of synchronizing data between the desktop computer and the corresponding handheld application. You implement the "body" of your conduit within this function. The following code is a sample of the `OpenConduit` function:

```
ExportFunc long OpenConduit(PROGRESSFN pFn, CSyncProperties& rProps)
{
    long retval = -1;

    // The pFn parameter is a function pointer to the PROGRESSFN
    // function, which
    // displays a status message during a HotSync operation.

    if (pFn)
    {
      //Begin the process of synchronization using the Sync Manager API
      //Implement the 'body' of your conduit within this function.
    }
    return(retval);
}
```